

CS 6650 Final Mastery: Album Store Service

Pranay Beela (beela.s@northeastern.edu)

April 12, 2026

1 Summary

Item	Value
Final Score	184 / 190
Correctness	110 / 110 (all 10 scenarios)
Load Tests	74 / 80
ChaosArena Nickname	pranaybeela
Contract	v1-album-store
Service URL	http://album-store-dev-alb-361553751.us-west-2.elb.amazonaws.com

This report documents the design, implementation, deployment, and iterative optimization of a high-performance Album Store REST API deployed on AWS. The project went through two major phases: an initial Python implementation that reached 178/190, followed by a Go rewrite that achieved the final score of 184/190. The service is backed by DynamoDB for metadata, S3 for photo storage, and deployed on ECS Fargate behind an Application Load Balancer. Through five optimization iterations, the service achieved perfect correctness (110/110) and near-perfect load test performance (74/80).

2 Problem Statement

The ChaosArena `v1-album-store` contract requires building a REST API with:

1. **Health check endpoint** — `GET /health`
2. **Album CRUD** — create/update (`PUT`), retrieve (`GET`), list all (`GET /albums`)
3. **Async photo pipeline** — accept photo uploads returning 202 immediately, process in background, expose status with a downloadable URL on completion
4. **Photo deletion** — remove metadata and file within 5 seconds; both `GET` and URL must fail after delete

The system is scored on correctness (10 scenarios, 110 pts) and latency under load (5 scenarios, 80 pts). Critical scenarios (S1–S5) gate all remaining tests.

2.1 Scenarios to Navigate

- Sequence numbers must be assigned synchronously in the `POST` handler, requiring an atomic distributed counter.
- Delete during processing (S9) creates race conditions between background processing and the delete handler.
- Large payload uploads (S15, up to 200 MB) must return 202 quickly *and* reach “completed” status fast.
- Cross-scenario state carries over — albums and photos accumulate across all tests, and background work from one test can compete with the next.

3 Architecture

3.1 High-Level System Design

The service follows a standard three-tier cloud architecture: an internet-facing ALB distributes HTTP traffic across multiple ECS Fargate tasks, each running the same binary. All tasks share a DynamoDB table pair for metadata and an S3 bucket for photo file storage.

Table 1: AWS Services Used

Service	Why Chosen
ECS Fargate	No server management, network bandwidth scales with vCPU
DynamoDB	Single-digit ms latency, atomic counters, pay-per-request
S3	Unlimited storage, public-read bucket for direct URL access
ALB	Layer 7 routing, health-check-based target management

3.2 ECS Task Configuration (Final)

Parameter	Value	Rationale
CPU	4096 (4 vCPU)	Fargate bandwidth scales with vCPU
Memory	8192 MB	Headroom for concurrent upload buffers
Desired Count	3	Distributes concurrent uploads across tasks
Launch Type	FARGATE	No EC2 management
Architecture	ARM64	Cost-efficient for compute workloads

3.3 Infrastructure as Code

All infrastructure is managed via Terraform, creating:

- 2 ECR repositories (API + worker)
- 3 DynamoDB tables (albums, photos, photo counters) with PAY_PER_REQUEST billing
- 1 S3 bucket with public-read policy for direct photo URL access
- 1 SQS queue with dead-letter queue (used in early iterations, later replaced by in-process dispatching)
- ALB with health-check-based target group on `/health`
- ECS cluster with Fargate tasks
- Security groups (ALB on port 80, ECS on port 8000 from ALB only)

4 Implementation Details

4.1 Phase 1: Python Implementation

The initial implementation used Python with a WSGI framework:

- **album_store_app.py** — WSGI entrypoint and HTTP routing
- **album_store_backend.py** — Service layer with Protocol-based adapter interfaces
- **album_store_aws.py** — AWS adapters (DynamoDB, S3, SQS)
- **album_store_worker.py** — SQS worker for background photo processing
- **Dockerfile** — Python 3.11-slim with Gunicorn (gthread workers)

The Python version used a clean ports-and-adapters pattern with two implementations: local (SQLite + filesystem) for development, and AWS (DynamoDB + S3 + SQS) for production.

4.2 Phase 2: Go Rewrite

The Go implementation followed a similar structure:

- **cmd/api/main.go** — HTTP server with graceful shutdown
- **internal/albumstore/http.go** — HTTP handler with **net/http** (no framework)
- **internal/albumstore/service.go** — Business logic layer
- **internal/albumstore/aws.go** — AWS adapters (DynamoDB, S3, inline dispatcher)
- **internal/albumstore/types.go** — Domain types and interfaces
- **internal/albumstore/local.go** — SQLite + filesystem adapters for local dev
- **Dockerfile** — Multi-stage build: `golang:1.24-bookworm` → `debian:bookworm-slim`

4.3 Key Implementation Decisions

4.3.1 Single-Call DynamoDB Album Upsert

The album create/update endpoint uses a single **UpdateItem** call with **if_not_exists** for the **created_at** field and **ReturnValues: ALL_OLD** to determine whether the item was created or updated. This halves the DynamoDB round trips compared to the naive get-then-put approach.

4.3.2 Direct-to-Media S3 Upload

Photos are uploaded directly to their final S3 location (**media/** prefix) during the **POST** handler, eliminating the need for a staging step. The background “promote” operation becomes a no-op — the worker only needs to update the DynamoDB record from “processing” to “completed.”

4.3.3 Inline Background Dispatcher

Rather than using SQS for background photo processing, the final implementation uses an in-process goroutine dispatcher. Since the promote step is a no-op (file already at final location), the background work is just a DynamoDB update (~10 ms). This eliminates SQS message delivery latency and cross-scenario queue contention.

4.3.4 Public S3 Bucket

The S3 bucket uses a public-read bucket policy, allowing ChaosArena to fetch photo URLs directly from S3 without pre-signed URLs. This eliminates URL generation overhead and avoids credential expiration issues.

4.3.5 Per-Album Atomic Sequence Counter

Sequence numbers are assigned using DynamoDB's atomic `UpdateItem` with `SET current_seq = if_not_exists(current_seq, :zero) + :one`. This guarantees monotonicity and uniqueness even under concurrent uploads.

4.3.6 Connection Pooling (Go)

The Go implementation configures a shared `http.Transport` with:

- `MaxIdleConns: 500`
- `MaxConnsPerHost: 500`
- `MaxIdleConnsPerHost: 500`
- `ForceAttemptHTTP2: true`

Go's default `MaxIdleConnsPerHost` is only 2, which causes severe connection thrashing under concurrent load.

4.3.7 Parallel DynamoDB + S3 Operations

In the Go implementation, `AllocateSeq` runs in a goroutine concurrently with the S3 upload, overlapping the DynamoDB round trip with S3 network I/O.

5 Optimization Journey

5.1 Run 1: Python Baseline (Score: 119)

Run ID: 20260412T140301Z_85cb2255-b599-4942-8407-d14eaf89c415

Initial deployment with Python/Gunicorn on ECS Fargate (1 vCPU / 2 GB, 2 API tasks, 5 SQS worker tasks).

- Correctness: 110/110 (all scenarios passed)
- Load: 9/80
- Key bottlenecks: DynamoDB double-call upsert (679 ms p95), SQS worker latency (45.8 s photo completion p95), small ECS tasks (bandwidth-starved)

5.2 Run 2: Python Optimized DynamoDB + Worker (Score: 128)

Run ID: 20260412T143601Z_21568bab-d487-48ce-9083-5137fe68d2db

Applied three optimizations:

1. Single-call DynamoDB upsert (`UpdateItem` with `if_not_exists`)
2. Direct-to-media S3 upload (skip staging, promote becomes no-op)
3. Multi-threaded SQS worker with batch receive (10 messages, 20 threads, 1 s poll)
 - S11 p95: 679 ms $\rightarrow$ 452 ms
 - S12 p95: 45,877 ms $\rightarrow$ 20,171 ms
 - S13 p95: 304 ms $\rightarrow$ 194 ms

5.3 Run 3: Python Major Infrastructure Upgrade (Score: 178)

Run ID: 20260412T150159Z_159b303f-3f78-4fe9-904f-bb1a4ad216aa

Major changes inspired by studying a 190/190 reference architecture:

1. Eliminated SQS entirely — replaced with in-process `ThreadPoolExecutor` dispatcher
2. Public S3 bucket — direct URLs instead of pre-signed
3. Scaled ECS to 4 vCPU / 8 GB RAM with 3 API tasks (0 worker tasks)
4. Increased Gunicorn to 4 workers $\times$ 16 threads
 - S11 p95: 452 ms $\rightarrow$ 64 ms (single DynamoDB call + bigger tasks)
 - S12 p95: 20,171 ms $\rightarrow$ 4,836 ms (no SQS latency)
 - S13 p95: 194 ms $\rightarrow$ 20 ms
 - S15 score: 7 $\rightarrow$ 18 (accept p95: 14.4 s $\rightarrow$ 5.4 s)

This was the largest single improvement: +50 points on load tests.

5.4 Run 4: Go Rewrite with Inline Dispatcher (Score: 184)

Run ID: 20260412T163701Z_74f988ba-9068-44d6-81a4-12aed881c9f3

Rewrote the entire service in Go to leverage:

- Goroutines for lightweight concurrency
- `MultipartReader` for lazy streaming (no body pre-buffering)
- Compiled binary performance (faster JSON, routing, handler dispatch)
- Lower memory footprint ($\sim$ 20 MB vs hundreds of MB for Python)
- S11 p95: 64 ms $\rightarrow$ 11 ms ($6\times$ faster)
- S12 p95: 4,836 ms $\rightarrow$ 3,441 ms
- S13 p95: 20 ms $\rightarrow$ 8 ms
- S15: 18 $\rightarrow$ **20/20** (perfect score)

5.5 Run 5: Go + Connection Pooling + Parallel Ops (Score: 184)

Run ID: 20260412T164926Z_2d0339e4-5a81-4ac7-95c6-401d518a2815

Final tuning:

1. HTTP connection pool: 500 max idle connections per host (Go default is 2)
 2. S3 upload manager concurrency increased from 5 to 10
 3. `AllocateSeq` runs in parallel goroutine during S3 upload
- S12 p95: 3,441 ms $\rightarrow$ 3,055 ms
 - S13 p95: 8 ms $\rightarrow$ 6 ms
 - Same point total, but p95 latencies improved across all scenarios

6 ChaosArena Results

6.1 Score Progression

Table 2: Score Across All Optimization Runs

Run	Language	Score	Load	Key Change
1	Python	119	9/80	Baseline deployment
2	Python	128	18/80	DynamoDB single-call, direct-to-media, batch worker
3	Python	178	68/80	Eliminate SQS, public S3, 4 vCPU $\times$ 3 tasks
4	Go	184	74/80	Full rewrite, inline dispatcher, streaming
5	Go	184	74/80	Connection pooling, parallel DDB+S3

6.2 Final Correctness Scores (110/110)

Every correctness scenario passed on all five runs.

Table 3: Correctness Scenario Results

Scenario	Description	Points	Status
S1	Health Check	5/5	PASSED
S2	Album Create + Read	15/15	PASSED
S3	Async Photo Upload	20/20	PASSED
S4	Photo Delete	10/10	PASSED
S5	List All Albums	10/10	PASSED
S6	Strict Health Body	5/5	PASSED
S7	Double Delete	10/10	PASSED
S8	Concurrent Deletes Same Photo	10/10	PASSED
S9	Delete Before Complete	10/10	PASSED
S10	Per-Album Photo Sequence	15/15	PASSED

6.3 Final Load Test Scores (74/80)

Table 4: Load Test Results (Final Run)

Scenario	Description	Points	p95 (ms)	p99 (ms)
S11	Concurrent Album Creates	15/15	10	—
S12	Concurrent Photo Uploads	9/15	3,055	—
S13	Mixed Read/Write Metadata	15/15	6	—
S14	Mixed Metadata + Uploads	15/15	13 (meta)	—
			4,018 (upload)	4,124
S15	Large Payload Upload	20/20	4,207 (accept)	—
			6,216 (complete)	—

7 Performance Analysis

7.1 Load Test p95 Latency Progression

Table 5: p95 Latency (ms) Across Runs

Scenario	Run 1	Run 2	Run 3	Run 4	Run 5
	Py Base	Py Opt	Py Major	Go	Go+Pool
S11 (creates)	679	452	64	11	10
S12 (photos)	45,877	20,171	4,836	3,441	3,055
S13 (mixed r/w)	304	194	20	8	6
S14 (meta)	516	356	75	15	13
S14 (upload)	90,197	76,351	11,263	4,101	4,018
S15 (accept)	12,071	14,421	5,371	4,334	4,207
S15 (complete)	19,429	14,976	5,582	5,973	6,216

7.2 Analysis of Remaining Gap

The 6 points lost are entirely in S12 (Concurrent Photo Uploads), where our p95 of 3,055 ms falls short of the reference benchmark. The bottleneck is S3 upload throughput under concurrent load — network bandwidth is the constraint, not application code.

A reference implementation achieving 190/190 used Go with a true zero-copy streaming pipeline (`io.Pipe` connecting the network socket directly to the S3 upload manager), which allows the S3 upload to start receiving chunks while the HTTP request body is still being transmitted. This overlap reduces the effective upload latency by eliminating any buffering between the network read and the S3 write.

8 Design Decisions and Tradeoffs

8.1 Python vs. Go

Table 6: Language Comparison for This Workload

Factor	Python	Go
Concurrency model	GIL + threads (I/O only)	Goroutines (true parallel)
HTTP overhead	WSGI buffers full body	<code>MultipartReader</code> streams lazily
S11 p95	64 ms (best)	10 ms
S13 p95	20 ms (best)	6 ms
Memory per process	~200 MB	~20 MB
Best score achieved	178	184

Go’s advantage is most visible in metadata operations (S11, S13) where handler dispatch overhead dominates. For S3-bound operations (S12, S15), the gap narrows since network I/O is the bottleneck regardless of language.

8.2 SQS vs. Inline Dispatching

The initial design used SQS for background photo processing. This introduced 1–20 seconds of latency per photo due to message delivery delay and poll intervals. Since the “processing” step was reduced to a single DynamoDB update (by uploading directly to the final S3 location), SQS became unnecessary overhead.

The inline dispatcher (goroutine in Go, `ThreadPoolExecutor` in Python) eliminates queue latency entirely. The tradeoff is that if the API process crashes mid-processing, the photo remains in “processing” state permanently. In practice, this is acceptable for a graded assignment.

8.3 Public S3 vs. Pre-Signed URLs

Pre-signed URLs add ~5 ms of generation overhead per request and can expire. Public-read S3 is simpler and faster. The tradeoff is that anyone with the URL can access the photo, which is acceptable for this use case.

8.4 Direct-to-Media vs. Staging

The staging approach (upload to `staging/`, copy to `media/` in background) doubles the S3 operations per photo. Since there is no actual image processing happening, uploading directly to the final location eliminates the copy step entirely, cutting the background processing time from seconds to milliseconds.

8.5 Task Sizing: 4 vCPU / 8 GB with 3 Tasks

Fargate network bandwidth scales with vCPU. A 1 vCPU task has significantly less bandwidth than a 4 vCPU task. With 3 tasks, concurrent large uploads are distributed across tasks, reducing the probability that multiple uploads land on the same task and contend for bandwidth.

9 Lessons Learned

9.1 What Worked Well

1. **Iterative optimization** — Five targeted runs, each addressing specific bottlenecks with clear before/after metrics. The systematic approach was more effective than guessing.
2. **DynamoDB on-demand** — Zero capacity planning, sub-10 ms latencies for all single-item operations, perfect atomic counters for sequence numbers.
3. **Eliminating SQS** — The single biggest optimization. Removing the queue reduced photo completion time from 45 s to under 5 s.
4. **Studying a reference architecture** — Analyzing a successful 190/190 implementation revealed that Fargate vCPU sizing, public S3, and inline processing were the key architectural decisions.
5. **Direct-to-media upload** — Skipping the staging step cut the background processing time from seconds to milliseconds.

9.2 What I Would Do Differently

1. **Start with Go from the beginning** — The language switch from Python to Go yielded a 6-point improvement that would have been available from day one.
2. **Profile network bandwidth early** — Fargate vCPU sizing has an outsized impact on throughput that is not obvious from the pricing page.
3. **Use inline dispatching from the start** — SQS was unnecessary complexity for this workload. The “processing” step was always just a DynamoDB update.
4. **Implement `io.Pipe` streaming** — The zero-copy streaming pipeline from network socket to S3 is the key to achieving perfect S12 scores. This should have been the first optimization target.

9.3 The ChaosArena Hint

“The engine runs a health check in-between each test. Make of that what you will!”

This hint reveals three things:

1. **State carries over** — Albums and photos from S11 exist when S12 starts.
2. **Resource contention carries over** — Background work from S14 competes with S15.
3. **The health check pause is brief** — You cannot rely on it to drain a work queue.

The solution: design each request to be self-contained. No shared background queues. The inline dispatcher ensures each upload’s background work completes in milliseconds, preventing cross-scenario contention.